Supplement: Computation, Optimization and Experiments for Auctions with Participant Audits

1 Overview

This supplement documents the code behind the numerical results: what it computes and how, how experiments are run, and what they output. The code is the Python package `audited_auctions` in the repository `audited-auctions-empirical` (branch `web27`). For a type distribution and a threshold function common to all bidders, the code does four things.

1. It computes the outcomes of four mechanisms exactly (sections 2 and 4).
2. It searches for the threshold function that maximizes exact expected welfare (section 5), on a gridded type distribution, and evaluates the result on a continuous version of that distribution (section 3).
3. It reports per-auction outcomes from simulated auctions and, for empirical data, bootstrap intervals and checks (section 6).
4. It runs whole experiments from configuration files and saves data, checks and figures (sections 7 and 8).

Notation follows the main paper.

2 Setting and Reported Quantities

There are n exchangeable bidders and $k < n$ identical goods with unit demand. Types (e_i, v_i) are independent draws from a distribution H with valuations $v_i \in [0, \bar{v}]$. All bidders use the same threshold function $\tau : \mathbb{R} \to [0, \bar{v}] \cup \{\bar{v}^+\}$: type (e, v) participates and bids v if $v \geq \tau(e)$, and abstains otherwise. The value $\tau(e) = \bar{v}^+$ excludes the externality e. The code compares the four mechanisms of table 1. Each is a score-based auction: the k participants with the highest positive scores win, ties are broken by a uniform lottery, and winners pay critical-value payments. So each auction without audits is DSIC and normalized.

Mechanism	Participants	Score	Penalty
VCG	every bidder	bid v	none
VCG-PA	$v \geq \tau(e)$	bid v	$r(e)$ inducing τ, paid by participants
IVCG-PA	$v \geq \tau(e)$	ironed welfare contribution $\bar{\omega}^\tau(v)$	$r(e)$ inducing τ, paid by participants
First best	every bidder	$v + e$	$-e$ per unit of allocation (recipient audit)

Table 1: The four mechanisms. VCG-PA and IVCG-PA share τ; their penalty functions differ because their interim allocations differ.

Penalties. The penalty is an output, not an input. With the interim allocation X^τ of the auction, the penalty that induces τ is

$$r(e) = \int_0^{\min\{\tau(e),\bar{v}\}} X^\tau(b)\, db.$$

An excluded externality, $\tau(e) = \bar{v}^+$, gets the penalty $\int_0^{\bar{v}} X^\tau(b)\, db$.

Outcomes of one auction. For allocation x, payments p and penalties paid r, the code reports six quantities. The first five are welfare $W = \sum_i (v_i + e_i)x_i$, its valuation part $\sum_i v_i x_i$, its externality part $\sum_i e_i x_i$, revenue $\sum_i p_i$ (auction payments only), and penalties $\sum_i r_i$. The sixth is bidders' net utility $\sum_i u_i$, defined by the identity

$$\textstyle\sum_i v_i x_i = \sum_i u_i + \sum_i p_i + \sum_i r_i, \qquad W = \sum_i v_i x_i + \sum_i e_i x_i.$$

In the first best, a winner faces the score $v + e$ and the penalty $-e$, so its payment is the critical score and the identity holds as well.

Comparisons. Every comparison is with VCG. The code reports four comparisons.

- The change over VCG of each quantity, $\mathbb{E}[Q_m] - \mathbb{E}[Q_{\text{VCG}}]$.
- The *share of the first-best gain*, $(\mathbb{E}W_m - \mathbb{E}W_{\text{VCG}})/(\mathbb{E}W_{\text{FB}} - \mathbb{E}W_{\text{VCG}})$. This is a ratio of expectations. The per-auction ratio is not used: the first best and VCG choose the same winners in many auctions, so its denominator is often zero.
- The relative revenue change $(\mathbb{E}\,\text{Rev}_m - \mathbb{E}\,\text{Rev}_{\text{VCG}})/\mathbb{E}\,\text{Rev}_{\text{VCG}}$.
- Per auction, the paired differences $Q_m - Q_{\text{VCG}}$ on the same type profile.

3 Type Distributions

A distribution is a finite list of cells. Each cell has an externality e_j, a valuation interval $[\ell_j, u_j]$ and a probability p_j, and the valuation is uniform on the interval. Atomic distributions have $\ell_j = u_j$. Continuous densities are represented as *piecewise uniform*: externalities on a grid of nodes, and, given the externality, a piecewise-constant density of valuations. All quantities are exact for this distribution (section 4). The discretization error relative to a continuous density shrinks with the grid; the grid-refinement check measures it (section 6).

Continuous versions. The grid has two kinds of jumps that a continuous density does not have: each externality node is a point mass in e, and the valuation density jumps at cell edges. Under a continuous τ, a whole node enters the participants at its threshold, and $\mathbb{E}[e \mid v]$ jumps wherever the mix of externalities changes. So the expected welfare contribution $\hat{w}^\tau$ drops at every node threshold and every cell edge, and IVCG-PA irons in many small bands. The welfare gain of IVCG-PA over VCG-PA from these bands shrinks with the squared spacing of the grid, so they are a discretization effect. The continuous version of a gridded distribution removes both jumps.

- *Externalities:* the mass of each node is uniform on its externality interval, from the midpoint to the previous node to the midpoint to the next.
- *Valuations:* for each node, the density is the piecewise-linear interpolation of the cell densities between cell midpoints, constant from the outer midpoints to the ends of a run of adjacent cells; cell masses are renormalized.

It differs from the grid by an error of second order in the spacing, the order of the grid's own error. Distributions with finitely many externality values, such as Example N, keep their externality values; atomic distributions have no continuous version. By default, the search uses the grid, and everything after it uses the continuous version (table 2).

Grid	Continuous version
search objective, best responses and gradients; first-order check; symmetric relaxation	exact outcomes, penalties, simulated auctions, bootstrap of smoothed resamples, IVCG-PA scores, ironing figure, maps of M^τ, sufficient-condition check

Table 2: Distribution used at each stage (default; option `continuous = false` uses the grid throughout).

Synthetic families. The synthetic families are listed below; their parameters are in the package documentation.

- `harm_rises`: a correlated bivariate normal; harm grows with valuation.
- `harmful_middle`: harm grows steeply with valuation, as in `harm_rises`, and the conditional mean $\mathbb{E}[e \mid v]$ has a wiggle in the middle, where it falls faster than v rises. The expected welfare contribution then decreases there, and ironing binds, also at optimized τ.
- `independent`: v and e independent.
- `heavy_tail`: lognormal valuations.
- Gaussian mixtures `a`–`f`, `h`.
- `example_n`: Example N of the main paper, an analytic benchmark.

Every family has $v \in [0, \bar{v}]$. The default grid has 151 externality nodes and 150 valuation cells. On four families with $n = 8$ and $k = 2$, the welfare gains over VCG change by at most $1.1 \cdot 10^{-4}$ against a grid of 601×1200. The gains themselves are 0.06–0.5.

Empirical data. An empirical distribution comes from a table of scores, with type $(e, v) = (\zeta\, e^{\text{score}},\ s\, v^{\text{score}})$. Here the externality scale $\zeta > 0$ is the weight of one unit of externality score, and the valuation scale s is the value of one action. Rows with $v \leq 0$ are dropped. The tweet data have one row per tweet with a Community Notes note, 11,811 tweets. The two scores are built per tweet as follows.

- v^{score} is engagement actions (retweets, replies, likes, quotes, bookmarks) per 30-day month of age.
- e^{score} is the signed note signal per month. Each note counts $+1$ if classified not misleading and -1 if misleading, weighted by 1 plus the sum of its helpfulness ratings (1, $\frac{1}{2}$, 0).

The pipeline then prepares the data in five steps.

1. *Split.* The rows are split at random into a training half and a test half, with a fixed seed.
2. *Truncate.* Each half drops rows with valuation above a quantile of all valuations (default 0.995). This quantile is the bound $\bar{v}$.
3. *Design.* τ is optimized on the training half, and the IVCG-PA scores $\bar{\omega}^\tau$ are computed on its continuous version. By default the training half is kernel-smoothed: Gaussian kernel in coordinates $(\operatorname{asinh}(e/s_e), \log v)$ with Scott's bandwidth, variance-preserving.
4. *Evaluate.* The design is evaluated exactly on the test rows, used as atoms. Ironing on the evaluation rows themselves would rank each row by its true $v + e$ in sample, which is why the scores come from the training half.

5. *Bootstrap.* The test rows are resampled (section 6).

4 Exact Computation

Quantile representation. Let P be the participation probability. Participating types are ordered by bid. The participating quantile $q \in [0, P]$ is the participating mass below a bid. On each elementary bid interval (between cell ends and thresholds) or atom, the bid is linear in q. On the same interval, the expected welfare contribution $\hat{w}^\tau(b) = b + \mathbb{E}[e \mid \text{bid } b, \text{ participate}]$ is linear in q as well.

Continuous versions. The threshold function is evaluated at the ends and midpoint of each cell's externality interval and at its kinks and jumps there (knots, nodes, bin edges, the exclusion cutoff), and taken as linear in between; this is exact for the piecewise-linear forms of table 3. At bid b, the participating externalities of a cell are then an interval whose ends are piecewise linear in b. So the participating density is piecewise quadratic in b, and $\mathbb{E}[e \mid \text{bid } b]$ is a ratio of polynomials. The bid line is cut at cell edges and midpoints and at the thresholds of the interval points. Each segment gets its exact participating mass with a constant bid density, and $\mathbb{E}[e \mid \text{bid } b]$ is computed exactly at the segment ends and interpolated linearly in between. Then $\hat{w}^\tau$ is continuous in the bid, and IVCG-PA irons only where $\hat{w}^\tau$ decreases. The computation is accurate to second order in the segment widths, instead of exact, and takes about five times as long. The first best uses a discrete approximation: four point externalities per interval and two uniform valuation pieces per cell.

Ironing. The IVCG-PA score $\bar{\omega}^\tau$ is the nondecreasing L^2 projection [Myerson, 1981] of $\hat{w}^\tau$ in quantile space, the derivative of the convex hull $\bar{\Psi}$ of $\Psi(q) = \int_0^q \hat{w}^\tau$. Pool-adjacent-violators computes it exactly for piecewise-linear $\hat{w}^\tau$, with pool ends inside segments found as roots of monotone equations. On a continuous version, $\hat{w}^\tau$ can decrease within a segment; such a segment is first replaced by its mean, which leaves the projection unchanged.

Interim allocation. Fix a participant with score c. Let a be the probability that a rival participates with a higher score, and z the probability that a rival participates with an equal score. If A rivals have higher scores and Z rivals tie, the participant wins with probability $\min\{1, (k-A)/(Z+1)\}$ when $A < k$ and $c > 0$, and 0 otherwise. Summing over the binomial distributions of A and Z gives X^τ. Without ties,

$$X^\tau = \Pr[\mathrm{Bin}(n-1, a) \le k-1],$$

a polynomial of degree $n-1$ in q.

Integration. Every expectation is an integral over segments of a polynomial of degree at most n, so Gauss–Legendre quadrature with $\lfloor n/2 \rfloor + 2$ nodes per segment is exact up to rounding. Expected welfare is $n \int \hat{w}^\tau X^\tau \, dq$; its valuation and externality parts are computed the same way. Expected revenue is

$$n\Big[\int b(q)\, X^\tau(q)\, dq - \int_0^\infty X^\tau(t) \Pr[\text{bid} > t]\, dt\Big].$$

The penalty paid is $n \sum_j \Pr[\text{cell } j, \text{ participate}]\, r(e_j)$; on a continuous version, the sum runs over the participating parts of the cells, each at its mean externality. VCG and the first best do not depend on τ, so their outcomes are computed once per distribution and market.

5 Choosing the Threshold Function

The objective is the exact expected welfare of the objective auction (VCG-PA or IVCG-PA) on the training distribution, on its grid. It is an exact number, not an average over simulated auctions, so it has no Monte Carlo noise. On the grid, externalities take finitely many values: grid nodes, or the distinct values of a sample. So only τ at those values matters, and the form `nodes` is the unrestricted problem. The other forms in table 3 restrict it. Between nodes, `nodes` interpolates linearly, which matters only on the continuous version.

Label	$\tau(e)$	Parameters	Methods
`poly<d>`	polynomial of degree d	$d+1$	genetic
`knots<K>`	continuous, piecewise linear between K knots at quantiles of e; $\tau = \bar{v}^+$ below a cutoff $\hat{e}$	$K+1$	all
`bins<K>`	constant on K quantile bins of e	K	all
`cutoff`	common welfare cutoff w: $\max\{0, w-e\}$, and $\bar{v}^+$ if $w - e > \bar{v}$	1	genetic
`nodes`	one value per externality node	number of nodes	best response, relaxation

Table 3: Forms of the threshold function. A value at or above $\bar{v}$ excludes the externality.

Genetic search. A genetic algorithm runs over the parameters of a form: tournament selection, single-point crossover, Gaussian mutation scaled to each parameter's range. It stops after a maximum number of generations, or once the best welfare has risen by at most $10^{-4}(\mathrm{FB} - \mathcal{W}_{\mathrm{VCG}})$ over the last 60 generations, with $\mathrm{FB} - \mathcal{W}_{\mathrm{VCG}}$ the first-best gain on the training distribution. Nelder–Mead then refines from the best distinct candidates, in coordinates scaled to the search ranges. The genetic stage explores globally; Nelder–Mead converges locally. Independent restarts (default 4, with consecutive seeds) and the Nelder–Mead starts run in parallel, and the best result is kept.

Best-response ascent. Hold the auction and the rivals' thresholds fixed. Welfare is then linear in one bidder's participation set, with weights equal to the marginal contributions

$$M^\tau(e, v) = (v + e)\, X^\tau(v) - L^\tau(v),$$

where L^τ is the rival welfare loss. Let $g_e(t) = \mathbb{E}[\mathbb{I}(v \geq t)\, M^\tau(e, v) \mid e]$. Each iteration has three steps.

1. Compute X^τ and L^τ exactly, as Chebyshev series between breakpoints. L^τ comes from the rival with the k-th highest score, whom a winning bid displaces.
2. Compute one bidder's best response within the form, maximizing g_e.
   - For `nodes` and `bins`: exactly, over breakpoints, the exclusion value, and the zeros of $M^\tau(e, \cdot)$ where it changes sign from negative to positive.
   - For `knots`: by dynamic programming over pairs of adjacent knots, with the cutoff as a state.
3. Move all bidders by $\tau \leftarrow \tau + \alpha(\tau' - \tau)$, for the largest $\alpha \in \{1, \frac{1}{2}, \ldots, \frac{1}{64}\}$ that raises the exact welfare.

A best response moves one bidder while the common τ moves all n, so a fixed point of the ascent need not satisfy the first-order condition for the best common τ. A quasi-Newton polish (L-BFGS-B, at most 500 welfare evaluations per round) therefore follows, with the exact gradient $\partial\mathcal{W}/\partial\tau(e_j) = -n\,\pi_j\, f_j(\tau(e_j))\, M^\tau(e_j, \tau(e_j))$. Here π_j is the probability of the node e_j, and f_j is the density of v

given e_j. In the final τ of the `nodes` form, a threshold in $(0, \underline{v}_j]$, where $\underline{v}_j$ is the lowest valuation of node e_j, is set to 0: the participants and the welfare are the same, and the penalty $r(e_j) = \int_0^{\tau(e_j)} X^\tau$ is the smallest.

For IVCG-PA, each step holds the scores $\bar{\omega}^\tau$ of the current τ fixed. This is justified at a joint optimum of the threshold function and the auction (main paper, section on the joint choice). Elsewhere, that the resulting derivative is the derivative of the IVCG-PA welfare $\mathcal{W}^*(\tau)$, whose scores change with τ, is not proved; numerically, it agrees with finite differences of $\mathcal{W}^*$.

The default starts, run in parallel, are the following.

- $\tau \equiv 0$.
- The zero-welfare threshold $\tau^0(e) = \max\{0, -e\}$, which is the first best without competition.
- A short genetic search in the same form.
- The threshold function of the symmetric relaxation (below), for piecewise-uniform distributions and $k < n$.
- Three random threshold functions, piecewise linear through 6 externalities, with values uniform on $[0, q_{0.95}]$ for $q_{0.95}$ the 0.95 quantile of v.

Across externality scales, an optional start is the previous optimum, rescaled: $\tau(e) = \tau_{\text{prev}}(e\,\zeta_{\text{prev}}/\zeta)$. The best result over the starts is kept. Both optimizers find local optima.

Symmetric relaxation. A symmetric participation rule lets each type participate with a probability $\alpha(e, v)$, the same for all bidders; an equal threshold function is the rule $\mathbb{I}(v \geq \tau(e))$. For VCG-PA with k identical goods, expected welfare over all such rules is a concave program (main paper, proof of the sufficient condition for VCG-PA), and its value bounds the VCG-PA welfare of every equal threshold function from above. On valuation intervals (600 by default), with tangent cuts for the concave allocation term, it is a linear program, solved in about 5 seconds per distribution and market. The result is a midpoint-rule estimate, with error of order (interval width)^2, not a certified bound. A threshold function is read off the solution: each node's threshold is the first valuation at which at least half of the node participates, and 0 if that is the node's lowest valuation. If the participating fraction of every node is nondecreasing in v, the best symmetric rule is a threshold rule, and that threshold function is a best equal threshold function for VCG-PA, up to the discretization. The method `relaxation` runs best-response ascent from this threshold function alone and records the program's value and whether its solution is a threshold rule. The program is convex, so its solution does not depend on a start; it is not faster than best-response ascent on `nodes`, which converges in 0.1–1.1 seconds per start.

Stopping and diagnostics. Each stage records why it stopped: convergence (no improving step, relative gain below 10^{-10} in two consecutive iterations, stalled genetic search, or the polish's gradient tolerance), an iteration cap, or the time limit. The time limit (default 60 minutes per combination, over all stages and starts) ends each stage after its current iteration, skips later starts, and keeps the best τ found. At every iteration of best-response ascent, the code records three measures at the current τ, each zero at a stationary point of the form or an optimum of the common τ.

- *Stationarity:* $m \sum_j |\partial\mathcal{W}/\partial\theta_j|/(\text{FB} - \mathcal{W}_{\text{VCG}})$, the projected gradient in the form's parameters θ, with m the median valuation. Welfare has kinks where thresholds cross valuation-cell edges, so a restricted form can be optimal at a kink with a small positive stationarity.
- *First-order violation:* the largest, over checked nodes, of $|M^\tau(e, \tau(e))|/\bar{v}$ at interior thresholds, $\max\{0, -M^\tau(e, 0)\}/\bar{v}$ at $\tau(e) = 0$, and $\max\{0, M^\tau(e, \bar{v})\}/\bar{v}$ at exclusion. A node is checked if moving its threshold in the improving direction changes welfare: the mass of the node times the density of v given e on that side of $\tau(e)$, times $|M^\tau|$, exceeds 10^{-12}. Unchecked nodes have $\tau(e)$

in a gap of the valuation support, where welfare is flat in $\tau(e)$, or negligible mass.
- *Best-response gap:* $\sum_e \big(\max_t g_e(t) - g_e(\tau(e))\big)/(\mathrm{FB} - \mathcal{W}_{\mathrm{VCG}})$, the gain of one bidder deviating alone. A common optimum need not be a best response, so the gap need not vanish there.

Benchmarks of the paper. Three threshold functions of the main paper are available as benchmarks. The zero-welfare threshold τ^0 is the welfare cutoff $w = 0$. The best common welfare cutoff is the form `cutoff`, found by genetic search. The cutoff τ^* of the ex ante relaxation for k identical goods has $\Pr[v + e > w^*] = \min\{k/n, \Pr[v + e > 0]\}$. For τ^*, IVCG-PA attains at least $\max\{\beta_{n,k}, 1 - 1/e\}$ of the first-best welfare, with $\beta_{n,k} = 1 - \binom{n}{k}(1 - \frac{k}{n})^{n+1-k}(\frac{k}{n})^k$; the code checks this guarantee.

Decomposition of the welfare gap. Following the main paper, each run reports, per distribution, externality scale, market and class $\mathcal{T}$ of thresholds, three gaps:

$$\mathrm{FB} - \mathrm{OPT}, \qquad \mathrm{OPT} - \mathrm{OPT}^{\mathrm{VCG}}, \qquad \mathrm{OPT}^{\mathrm{VCG}} - \mathrm{OPT}^{\mathrm{VCG}}_{\mathcal{T}}.$$

These measure, in turn, the loss from not observing externalities, the loss from ranking by bid instead of by ironed welfare contribution, and the loss from restricting thresholds to $\mathcal{T}$. Here $\mathrm{OPT} = \sup_\tau \mathcal{W}^*(\tau)$, $\mathrm{OPT}^{\mathrm{VCG}} = \sup_\tau \mathcal{W}^{\mathrm{VCG}}(\tau)$, and $\mathrm{OPT}^{\mathrm{VCG}}_{\mathcal{T}}$ is the supremum over $\mathcal{T}$. The code replaces each supremum by the best value found among the run's thresholds. So OPT and $\mathrm{OPT}^{\mathrm{VCG}}$ are lower bounds, and they are close to the suprema only when the run includes the unrestricted form `nodes`.

6 Uncertainty and Checks

Two kinds of spread. The variation of an outcome across auctions is a property of the mechanism. Simulated auctions show it: type profiles are drawn from the evaluation distribution, and every mechanism runs on the same draws (common random numbers). Allocations are averaged over the tie-breaking lottery, and payments are critical values given the realized rival bids. The uncertainty of an expected value has two cases.
- *Synthetic families:* none. The distribution is known and every expectation is exact.
- *Empirical data:* the test rows are resampled with replacement, and every expected outcome is recomputed exactly for each resample (on the continuous version of each resample when the test half is smoothed). The reported intervals are percentile intervals, for levels, for changes over VCG (paired by resample), and for shares. The design (τ and the IVCG-PA scores) is held fixed, so the intervals do not include the uncertainty of the design.

Checks. Every combination of a run gets the checks in table 4, each with a value, a tolerance and a verdict.

The first-order check rests on two results of the main paper: at an optimal interior threshold the threshold type has zero marginal contribution, and the penalty equals the Pigouvian charge $-eX^\tau + L^\tau - P^\tau$. The check reports penalty, charge and residual at every externality node.

7 Running Experiments

A run is described by a TOML file and started with `python run.py CONFIG.toml`. Every list in the file is a set of values, and the run computes every combination; combinations that cannot run, such as the relaxation with $k \geq n$, are skipped and listed. There are two kinds of run.

Check	Value	Verdict
Monte Carlo	largest $\lvert z\rvert$ of a simulated mean (welfare, revenue) against its exact value	pass if < 4
grid refinement	largest relative change of a gain over VCG on a twice finer grid, τ fixed (synthetic; continuous versions of both grids by default)	pass if $< 1\%$
first order	first-order violation (above), on the training grid	pass if $< 10^{-4}$ for `nodes`; for restricted forms, the distance from the per-node optimum
starts	spread of the welfare reached from different starts, in units of $\mathrm{FB} - \mathcal{W}_{\mathrm{VCG}}$	information: local optima
search stop	number of search stages stopped by the time limit or an iteration cap	pass if all converged; information at an iteration cap; fail at the time limit
bandwidth	spread of the gain over VCG when τ is redesigned with bandwidth multipliers (0.5, 2)	pass if at most the width of the gain's bootstrap interval
ex ante guarantee	$\mathcal{W}^*(\tau^*)/\mathrm{FB}$ for the ex ante cutoff, on the training distribution	pass if at least $\max\{\beta_{n,k}, 1 - 1/e\}$
sufficient condition	$G(\tau)/(\mathrm{FB} - \mathcal{W}_{\mathrm{VCG}})$ for VCG-PA, $G(\tau) = \mathbb{E}[M^{\tau-}\,\mathbb{I}(\text{participate}) + M^{\tau+}\,\mathbb{I}(\text{abstain})]$, the largest welfare gain from one bidder changing its participation set	information; 0 if and only if τ is best among symmetric participation rules for VCG-PA
symmetric relaxation	$(\mathcal{W}^{\mathrm{VCG}}(\tau) - \mathcal{W}_{\mathrm{VCG}})/(J - \mathcal{W}_{\mathrm{VCG}})$ on the training grid, J the value of the symmetric relaxation	information; values slightly above 1 are discretization error

Table 4: Checks of a combination (`checks.csv`).

- `optimize`: the sets are distributions (each with its externality scales ζ), numbers of bidders n and of goods k, objectives, and optimizers. Each optimizer has a method (`genetic`, `best_response`, `genetic+best_response`, `relaxation`), its parameters, and its own list of threshold forms; a form the method cannot search is a configuration error.
- `evaluate`: the sets are distributions, ζ, n, k, and given thresholds. A threshold is given as a polynomial, a constant, a tabulated function, a welfare cutoff (a number, or the ex ante cutoff), a saved `tau.json`, or every optimum of an earlier run.

The option `continuous` (default true) selects evaluation on continuous versions (table 2). Before running, the code prints the plan and a time estimate, calibrated by timing exact evaluations on each distribution. Combinations run in parallel worker processes (default: number of cores minus 2). Workers beyond one per combination run the parallel stages within a combination: search starts and restarts, Nelder–Mead starts, and the bootstrap. Table 5 lists the command-line options.

Reproducibility. The run record `run.toml` stores the following.

- The resolved configuration, with every default explicit.
- The random seeds: train/test split, search, simulation, bootstrap.
- The git commit, and whether tracked files had uncommitted changes. If they had, their diff is saved as `code.patch`.

Command	Does
`run.py CONFIG.toml`	plan, time estimate, confirmation, computation, figures, README files
`--dry-run`	plan and time estimate only
`--yes`	no confirmation (batch jobs)
`--workers N`	number of worker processes
`--no-report`	computation only, no figures
`run.py --report RUN`	figures and README files of an existing run, from its saved files
`--paper [NAMES]`	figures for a paper: one PDF per panel (section 8)
`run.py --options`	distributions, forms, objectives, optimizers, threshold sources and all defaults

Table 5: Command-line options of `run.py`.

- Package versions; the host (local machine, or the BU SCC cluster with its job id); the SHA-256 hash of each data file.
- Start and end times, and the runtime of each stage.

Rerunning the saved `config.toml` at the recorded commit reproduces the run. Figures are drawn only from saved files, so `python run.py --report RUN_FOLDER` redraws them without recomputation.

Test runs. Two configurations test the code.

- `configs/smoke_test.toml`: 6 combinations through every stage on coarse grids, about 15 seconds; for code changes.
- `configs/standard_test.toml`: every feature, about three minutes. It has two distributions: `harmful_middle`, and the tweet data at $\zeta \in \{5, 50\}$. It runs $n \in \{4, 8\}$, $k \in \{1, 2\}$, both objectives, and three optimizers: genetic search on `poly1`, `knots4` and `cutoff`, best-response ascent on `knots4`, and the relaxation on `nodes`, with short searches on coarse grids: 120 combinations.

BU SCC cluster. The same runs execute as batch jobs on the BU Shared Computing Cluster. The helper `scc/scc.sh`, run on the local machine, does the following.

- `submit CONFIG SLOTS HOURS`: push the branch, pull it on the cluster, print the plan, and submit the run with `SLOTS` worker processes and a run-time limit of `HOURS`. The cluster runs the last pushed commit.
- `status`, `log`: queue state; end of the newest job log.
- `fetch [RUN]`: copy run folders back, and merge the cluster's lines of `index.csv` into the local one.

Run folders on the cluster are on project storage; `run.toml` records the job id and slots.

8 Outputs

A run writes one folder, `<date>_<time>_<kind>_<name>`, with the files of table 6. Each combination has a sub-folder

`<distribution>/z<ζ>/n<n>-k<k>/<design>`.

The design label is `<objective>_<form>_<optimizer>` in optimize runs and `tau-<label>` in evaluate runs. A comparison figure goes in the `compare/` folder of the deepest level above every input it varies.

File	Contents
`README.md`	index: inputs, counts of check verdicts, results table with links to every figure, gap decomposition, figure reference
`config.toml`, `run.toml`	configuration as given; resolved configuration and provenance
`code.patch`, `log.txt`	uncommitted changes of tracked code (if any); progress messages
`summary.csv`	one row per combination: inputs, exact outcomes, gains, shares, revenue changes, bootstrap bounds, runtime
`checks.csv`	one row per check and combination
`gap-decomposition.csv`	the three gaps per distribution, ζ, market and class of thresholds; the relaxation value J and its gap to the best VCG-PA welfare found, with the distribution on which both are measured
`tau.json`, `tau.csv`	τ (rebuildable, reusable in evaluate runs); $\tau(e)$ and both penalty functions at the externality nodes
`results.json`	every statistic of a combination, including bootstrap intervals, search statistics, convergence trace, the relaxation record and the sufficient-condition measures
`first-order.csv`	per node and mechanism: threshold, X^τ, penalty, Pigouvian charge, marginal contribution, best-response gap, first-order violation
`data.npz`	arrays behind the figures: simulated outcomes, bootstrap resamples, ironing segments, maps of M^τ, tried thresholds
`types.npz`	per distribution and ζ: the type distribution drawn behind the figures
`README.md` (combination)	results table, files, each figure with its description and preview, figures not drawn and why
`runs/index.csv`	one line per run: folder, kind, start, runtime, combinations, failures, failed checks, commit, host

Table 6: Files of a run and of each combination.

Previews and README files. Every figure has a WebP preview at 45 pixels per inch in a `thumbs/` folder next to it. The README file of each combination and of each `compare/` folder shows the previews, each linked to its PDF and preceded by a description. Dense parts of the PDFs (scatter points, maps, tried thresholds) are embedded as images at 150 dpi; axes, text and curves stay vector.

Figures for a paper. The option `--paper` exports each panel of the selected figures as its own PDF at printed size (3.1×2.4 inches for panels of multi-panel figures, 3.9×2.9 for single panels), with fixed margins, so panels placed side by side align. Legends and colorbars go to their own PDFs. Each figure also gets a caption draft and a LaTeX `figure` environment that includes the panels. The run's own figures stay unchanged; the README files link the exported panels.

9 Limitations

- *Discretization.* The search is exact for the piecewise-uniform distribution, which approximates a continuous density; τ is optimal for the grid. The evaluation on the continuous version is accurate to second order in the segment widths, and differs from the grid by the discretization error.
- *Local optima.* Both optimizers find local optima of a non-convex objective. Several starts, restarts and the check of their spread reduce the risk; they do not remove it.

Figure	Shows	Drawn
1 types, τ, penalties	type distribution, τ, abstention region, excluded externalities, first-best boundary $v = -e$, bids pooled by IVCG-PA; penalty functions and recipient penalty $-e$	always
2 expected outcomes	expected welfare as bidder net utility + revenue + penalties + externality part, with the share of the first-best gain; changes over VCG of each quantity, with intervals	always
3 outcome distributions	per quantity: per-auction distribution of each mechanism, and of the paired difference from VCG (with the share of exact zeros); exact means and intervals marked	always
4 gain scatter	per auction: captured gain against achievable gain; revenue change against welfare change	always
5 ironing	$\hat{w}^\tau$ and $\bar{\omega}^\tau$ in quantile space, Ψ and its convex hull, their gap, scores against bids, interim allocations	ironing binds
6 marginal contribution	maps of $M^\tau(e, v)$ with the zero contour and τ; penalty against Pigouvian charge; M^τ at threshold types	always
search: tried	tried thresholds behind the best one	optimize
search: convergence	training objective against time, per start and stage	optimize
search: iterations	per iteration: share of the first-best gain, distance to the best welfare found, stationarity, first-order violation, best-response gap	optimize
compare: τ	τ of combinations that differ in one input (market, form, optimizer, objective, threshold)	≥ 2 such
compare: ζ	welfare and gains over VCG against ζ	several ζ
compare: share	share of the first-best gain of every combination	always

Table 7: Figures. All are PDF. Each mechanism has a fixed color and line style, chosen to remain distinguishable under color-vision deficiencies.

- *Time limit.* A search stopped by the time limit returns the best τ found so far; the check `search_stop` fails for that combination.
- *IVCG-PA objective.* Best-response ascent for IVCG-PA holds the scores fixed in each step. This is proved valid at a joint optimum only, and the line search on exact welfare protects each step.
- *Bootstrap.* The intervals cover the sampling of the test population, not the design: τ is not re-optimized within resamples.
- *Valuation bound.* The truncation quantile sets $\bar{v}$, and with it the penalty for excluded externalities.

References

Roger B. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981. ISSN 0364-765X. URL https://www.jstor.org/stable/3689266.